Final Project

Ishay Ishon, Matan Brimer, Tair Mazriv
Supervised by: Dana Shapira

August 2026

1 Dynamic Programming and the Knapsack Problem

1.1 Introduction

Before addressing the jobs scheduling problem, we first studied the classical 0/1 Knapsack problem. The purpose of this stage was to gain familiarity with Dynamic Programming (DP) techniques, which later serve as a foundation for solving more advanced optimization and scheduling problems.

1.2 Problem Definition

The 0/1 Knapsack problem is defined as follows:
Given:

- A knapsack with maximum capacity W
- A set of n items
- Each item i has:
  - Weight w_i
  - Value v_i

The objective is to select a subset of items such that:

- The total weight does not exceed the knapsack capacity
- The total value is maximized

The mathematical formulation is:

$$\max \sum_{i=1}^{n} v_i x_i$$

subject to:

$$\sum_{i=1}^{n} w_i x_i \leq W$$

where:

$$x_i \in \{0, 1\}$$

The binary variable x_i indicates whether item i is selected.

1.3 Dynamic Programming Approach

A Dynamic Programming solution is based on solving overlapping subproblems and storing intermediate results.

We define:

$$dp[i][w]$$

as the maximum value obtainable using the first i items with knapsack capacity w.

For each item, two choices are considered:

1. Do not take the item:

$$dp[i][w] = dp[i-1][w]$$

2. Take the item (only if its weight does not exceed the current capacity):

$$dp[i][w] = v_i + dp[i-1][w - w_i]$$

Therefore, the recurrence relation is:

$$dp[i][w] = \max\left(dp[i-1][w], v_i + dp[i-1][w - w_i]\right)$$

This recurrence allows us to build the solution incrementally until the optimal solution is obtained.

1.4 Algorithm

The following pseudocode describes the Dynamic Programming algorithm:

```
Initialize dp table with zeros

for i = 1 to n:
    for w = 1 to W:
        if weight[i] <= w:
            dp[i][w] = max(
```

```
                value[i] + dp[i-1][w-weight[i]],
                dp[i-1][w]
            )
        else:
            dp[i][w] = dp[i-1][w]

return dp[n][W]
```

1.5 Complexity Analysis

The algorithm fills a table of size $(n + 1) \times (W + 1)$.

1.5.1 Time Complexity

The algorithm iterates over:

- All n items
- All capacities from 1 to W

Therefore, the time complexity is:

$$O(nW)$$

1.5.2 Space Complexity

The DP table stores $(n + 1)(W + 1)$ entries, resulting in:

$$O(nW)$$

space complexity.

1.6 Example

Consider the following example:

- Maximum capacity: $W = 8$
- Weights: $\{2, 3, 4, 5\}$
- Values: $\{3, 4, 5, 8\}$

The optimal solution is obtained by selecting the items with weights 3 and 5, resulting in:

$$3 + 5 = 8$$

total weight and:

$$4 + 8 = 12$$

total value.
Therefore, the maximum obtainable value is:

12

1.7 Connection to the Scheduling Problem

The Knapsack problem demonstrates how Dynamic Programming can efficiently solve optimization problems involving combinatorial decisions under constraints.

This idea is closely related to our scheduling problem, where decisions must be made regarding:

- Which jobs should be scheduled
- Which jobs should be rejected

while minimizing the total rejection weight.
Studying the Knapsack problem provided an important introduction to:

- Dynamic Programming techniques
- State representation
- Recurrence relations
- Optimization under constraints

which are fundamental concepts used later in the project.

2 Naive Solution for the Proportionate Flow Shop Scheduling Problem

2.1 Introduction

After studying the Knapsack problem and Dynamic Programming techniques, we moved to the basic version of the scheduling problem.

In this problem, a set of jobs must be scheduled in a proportionate flow shop with M machines. Each job passes through all machines in the same order, and the processing time of job j is identical on each machine, that is, $p_{ij} = p_j$. Each job has:

- A processing time p_j
- A weight w_j

The objective is to find an ordering of the jobs that minimizes the total weighted completion time.

2.2 Problem Definition

Given:

- A set of N jobs
- M machines
- For each job j:
  - Processing time p_j
  - Weight w_j

For a given schedule σ, let $C_j(\sigma)$ denote the completion time of job j.
The optimization objective is:

$$\min \sum_{j=1}^{N} w_j C_j(\sigma)$$

The goal is therefore to minimize the weighted sum of completion times.

2.3 Completion Time Formula

For a proportionate permutation flow-shop schedule on M machines, the completion time of job j can be calculated using:

$$C_j(\sigma) = \sum_{k=1}^{j} p_k + (M-1)\max\{p_1, p_2, \ldots, p_j\}$$

This formula was verified experimentally in our implementation by comparing it with the full machine-by-machine simulation.

2.4 Naive Exhaustive Search Approach

The implemented solution is a naive exhaustive-search algorithm.

The algorithm generates all possible permutations of the jobs and computes the weighted completion time for each permutation. The permutation with the minimum total weighted completion time is selected as the optimal solution.

Since every possible ordering is examined, this approach guarantees finding the optimal solution.

2.5 Algorithm

The following pseudocode describes the naive algorithm:

```
bestValue = infinity

Generate all permutations of the jobs

for each permutation:

    Compute completion times

    totalWeight = 0

    for each job:
        totalWeight += weight[j] * completionTime[j]

    if totalWeight < bestValue:
        bestValue = totalWeight
        save current permutation

return best permutation
```

2.6 Completion Time Calculation

For each permutation, the completion times are computed using a dynamic table.

Let:

$$completion[i][m]$$

represent the completion time of job i on machine m.

The recurrence relation is:

$$completion[i][m] = \max\left(completion[i-1][m], completion[i][m-1]\right) + p_i$$

This guarantees that:

- A machine can process only one job at a time
- A job must complete on machine $m-1$ before starting on machine m

2.7 Example

Consider the following example:

- Number of machines: $M = 3$
- Jobs:

Job	$ProcessingTime$	$Weight$
1	4	3
2	2	1
3	6	2
4	3	4

The algorithm generates all possible job permutations:

$$4! = 24$$

possible schedules.

For each schedule, the weighted completion time is computed, and the permutation with the minimum value is selected.

2.8 Complexity Analysis

2.8.1 Time Complexity

The algorithm checks all possible permutations of the jobs.

The number of permutations is:

$$N!$$

For each permutation, completion times are calculated for all jobs and all machines, requiring:

$$O(NM)$$

operations.

Therefore, the overall time complexity is:

$$O(N! \cdot N \cdot M)$$

This complexity grows extremely fast and becomes impractical even for relatively small numbers of jobs.

2.8.2 Space Complexity

The completion table stores:

$$N \times M$$

values.

Therefore, the space complexity is:

$$O(NM)$$

2.9 Discussion

The naive exhaustive-search solution guarantees an optimal result because every possible job ordering is examined.

However, due to the factorial growth of the number of permutations, this approach is computationally expensive and unsuitable for large-scale instances.

This limitation motivates the need for more efficient optimization techniques, including Dynamic Programming and advanced scheduling algorithms, which are explored in the following stages of the project.

3 Dynamic Programming Attempt

After implementing the naive exhaustive-search algorithm, we attempted to improve its running time using Dynamic Programming (DP). The main idea was to avoid examining all possible job permutations by storing solutions to smaller subproblems and reusing them when constructing larger solutions.

We first considered a subset-based DP, in which each state represents a subset of scheduled jobs. Although this approach reduces the factorial running time of the naive solution, it still requires an exponential number of states, making it impractical for large instances. We also considered using a more compact state based only on aggregate information, such as the total and maximum processing times. However, in the multi-machine setting, the order of the jobs directly affects their completion times and the objective value. Therefore, such a compact state does not preserve enough information to determine the optimal continuation of a partial solution.

Thus, within the scope of this project, we did not find a polynomial or pseudo-polynomial Dynamic Programming formulation that guarantees an optimal solution for the general multi-machine scheduling problem. This motivated us to search for a more efficient polynomial-time scheduling algorithm, which is presented in the following section.

4 Improving the Scheduling Algorithm Using WSPT-MCI

4.1 Motivation

After implementing the naive exhaustive-search solution and the subset-based Dynamic Programming approach, we searched for a more efficient algorithm for the scheduling problem without job rejection.

The naive solution guarantees optimality, but its running time is factorial. The subset-based Dynamic Programming approach improves the complexity, but still requires exponential time and memory. Therefore, a polynomial-time algorithm is needed in order to handle larger instances.

At this stage, we used the WSPT-MCI algorithm, which appears in:

Michael L. Pinedo, *Scheduling: Theory, Algorithms, and Systems*, Fifth Edition, Springer, 2016.

In Pinedo's book, this algorithm is presented as Algorithm 6.3.4. Theorem 6.3.5 states that WSPT-MCI generates an optimal schedule for the problem

$$F_m \mid prmu,\ p_{ij} = p_j \mid \sum w_j C_j.$$

This is the proportionate permutation flow shop problem, where the processing time of job j is identical on all machines, and the objective is to minimize the total weighted completion time.

Therefore, unlike the heuristic algorithms that will be introduced later for the problem with job rejection, in this section WSPT-MCI is used as an exact polynomial-time algorithm for the scheduling problem without rejection.

4.2 Problem Setting

In this section, we focus on the scheduling problem without job rejection. All jobs must be processed, and the goal is to find a permutation of the jobs that minimizes

$$\sum_j w_j C_j.$$

The model is a proportionate flow shop with M machines. Each job passes through all machines, and its processing time is the same on every machine:

$$p_{ij} = p_j.$$

Thus, if job j has processing time p_j, then it requires the same processing time on each of the M machines.

The goal is to determine a permutation schedule that minimizes the total weighted completion time.

4.3 The WSPT Rule and Its Role

The WSPT rule, which stands for Weighted Shortest Processing Time, sorts jobs in non-increasing order of the ratio

$$\frac{w_j}{p_j},$$

where:

- w_j is the weight of job j,
- p_j is the processing time of job j.

For a single machine, the WSPT rule gives an optimal order for minimizing

$$\sum_j w_j C_j.$$

However, as discussed in the previous section, for the multi-machine proportionate flow shop, the WSPT order alone is not guaranteed to be optimal.

Therefore, the algorithm in this section does not simply sort the jobs according to WSPT and return that order. Instead, the WSPT order is used as the order in which jobs are considered, and each job is inserted into the best position within the partial sequence constructed so far.

This is the idea behind WSPT-MCI:

$$Weighted Shortest Processing Time first with Minimum Cost Insertion.$$

4.4 WSPT-MCI Algorithm Description

The algorithm operates as follows:

1. Sort all jobs in non-increasing order of w_j/p_j.
2. Start with a partial sequence containing only the first job in the WSPT order.
3. At iteration k, take the next job according to the WSPT order.
4. Try inserting this job into every possible position in the current partial sequence.
5. For each possible insertion position, compute the resulting value of the objective function
$$\sum_j w_j C_j.$$
6. Choose the insertion position that causes the smallest increase in the objective function.
7. If several insertion positions lead to the same minimum increase, choose the latest such position, as described in Pinedo's algorithm.
8. Continue until all jobs have been inserted into the sequence.

At the end of the algorithm, a complete permutation of all jobs is obtained.

According to Theorem 6.3.5 in Pinedo, the sequence generated by WSPT-MCI is optimal for

$$F_m \mid prmu,\ p_{ij} = p_j \mid \sum w_j C_j.$$

4.5 Completion Time Evaluation

In a proportionate flow shop, the completion time of the job in position j of a given sequence is calculated by

$$C_j = \sum_{k=1}^{j} p_k + (M-1)\max\{p_1, p_2, \ldots, p_j\}.$$

The sum and the maximum are taken over the jobs that appear up to position j in the sequence.

It is important to note that this is not an approximation in our setting. The formula follows from the structure of the proportionate flow shop, where each job has the same processing time on all machines.

Thus, the completion time depends on two components:

- the total processing time of the jobs up to the current position,
- the maximum processing time among the jobs up to the current position, multiplied by the additional $M-1$ machines.

Using this formula, we can evaluate the objective function

$$\sum_j w_j C_j$$

for any given sequence.

4.6 Algorithm Complexity

The algorithm inserts the jobs one by one.

When the current partial sequence contains k jobs, inserting the next job requires checking up to $k+1$ possible insertion positions.

Using auxiliary arrays to evaluate insertion costs efficiently, each iteration can be performed in linear time in the size of the current partial sequence.

Therefore, summing over all iterations gives

$$\sum_{k=1}^{n} O(k) = O(n^2).$$

Hence, the total running time is

$$O(n^2).$$

This is a significant improvement over the naive exhaustive-search solution, whose running time is

$$O(n! \cdot n \cdot M),$$

and over the subset-based Dynamic Programming approach, whose running time is

$$O(n \cdot 2^n).$$

In terms of space complexity, the algorithm stores the current partial sequence and several auxiliary arrays of size at most n. Therefore, the space complexity is

$$O(n).$$

4.7 Implementation Validation

The optimality of WSPT-MCI for the scheduling problem without rejection follows from the theoretical result in Pinedo's book, rather than from our experiments.

However, we implemented the algorithm and compared its results against the naive exhaustive-search solution on small instances, where all permutations can still be enumerated. The purpose of this comparison was to validate our implementation.

For small instances, the naive algorithm serves as an implementation check, since it examines all possible job orders and therefore finds the optimal solution.

4.8 Discussion

This stage represents the transition from exact but inefficient methods, such as exhaustive search and exponential subset-based DP, to an exact polynomial-time algorithm for the scheduling problem without job rejection.

The important point is that WSPT alone is not sufficient for the multi-machine case. However, WSPT-MCI uses the WSPT order only as the order in which jobs are considered, and then applies a minimum-cost insertion step. According to Pinedo, this full algorithm generates an optimal sequence for the proportionate flow shop problem without rejection.

Later in the project, when job rejection is introduced, the problem becomes harder. It is no longer sufficient to optimally schedule a fixed set of jobs; the algorithm must also decide which jobs should be processed and which jobs should be rejected. Therefore, WSPT-MCI will be used as a basic scheduling subroutine for the set of jobs that are not rejected, while the rejection decision itself will require additional algorithms.

5 Scheduling with Job Rejection Constraints

5.1 Motivation

In the previous sections, we considered the scheduling problem in which all jobs must be processed. For the scheduling problem without rejection, we showed

that WSPT-MCI can be used to find an optimal sequence in polynomial time under the proportionate flow shop model.

In the next stage of the project, we extended the problem by allowing some jobs to be rejected. Each rejected job has a rejection cost, and the total rejection cost must not exceed a given rejection budget.

This extension makes the problem more complex, since the algorithm now has to make two types of decisions simultaneously:

1. Which jobs should be processed and which jobs should be rejected.
2. In which order the non-rejected jobs should be processed.

Even though WSPT-MCI can optimally schedule a given set of jobs, the set of jobs that should be processed is no longer known in advance.

5.2 Problem Definition with Rejection

We are given a set of n jobs. Each job j has:

- p_j: processing time,
- w_j: weight in the objective function,
- e_j: rejection cost.

In addition, we are given a total rejection budget R.

Each job can either be processed or rejected. Let J_R denote the set of rejected jobs, and let J_S denote the set of scheduled jobs.

The rejection budget constraint is

$$\sum_{j \in J_R} e_j \leq R.$$

The objective is to minimize the total weighted completion time of the scheduled jobs only:

$$\min \sum_{j \in J_S} w_j C_j.$$

Here, C_j is the completion time of job j according to the sequence of the scheduled jobs.

Rejected jobs are not included in the objective function. They only affect the feasibility of the solution through the rejection budget constraint.

5.3 Why WSPT-MCI Alone Is Not Sufficient

In the previous section, we showed that WSPT-MCI gives an optimal sequence for a given set of jobs in the proportionate flow shop without rejection.

However, in the problem with rejection, the set of scheduled jobs is not known in advance. The algorithm must decide which subset of jobs should be processed and which subset should be rejected.

Therefore, WSPT-MCI solves only part of the problem:

$$given a set of jobs to process, it can schedule this set optimally.$$

However, it does not determine which set of jobs should be selected.

For example, a job may have a large contribution to the objective function and therefore seem attractive to reject, but its rejection cost may be too high. On the other hand, another job may be cheap to reject but may have only a small effect on the objective function.

Thus, the rejection decision must take into account both the scheduling effect of a job and its rejection cost.

The problem with rejection therefore combines two decisions:

- selecting the subset of jobs to process,
- scheduling the selected subset.

5.4 Exact Baseline by Enumerating Rejection Sets

In order to obtain a reference solution for small instances, we implemented an exact baseline for the rejection decision.

The idea is to enumerate all possible subsets of jobs that may be rejected. For each rejection subset:

1. Compute the total rejection cost of the subset.
2. If the total rejection cost exceeds the budget R, discard the subset.
3. Otherwise, construct the set of jobs that are not rejected.
4. Schedule the non-rejected jobs using WSPT-MCI.
5. Compute the resulting objective value.
6. Keep the best solution found.

Since every feasible rejection subset is examined, and since WSPT-MCI optimally schedules the remaining jobs, this baseline gives an exact solution for small instances of the rejection problem.

5.5 Complexity of the Baseline

The number of possible subsets of n jobs is

$$2^n.$$

Therefore, enumerating all possible rejection sets is exponential.

For each feasible rejection subset, WSPT-MCI is applied to the remaining jobs. Since WSPT-MCI runs in

$$O(n^2)$$

time, the worst-case running time of the exact baseline is

$$O(2^n \cdot n^2).$$

Thus, this exact baseline is suitable only for small instances. For larger instances, it becomes computationally impractical.

5.6 NP-Hardness

To establish the computational complexity of our problem, we rely on the results presented by Mor and Shapira in *Scheduling with Regular Performance Measures and Optional Job Rejection on a Single Machine*. One of the authors of this work, Dana Shapira, is also the supervisor of this project. In their work, the authors show that the scheduling problem with optional job rejection on a single machine is NP-hard. Their problem is a special case of our problem. While Mor and Shapira consider scheduling on a single machine, our formulation allows the number of available machines to be greater than one. By setting the number of machines in our problem to (m=1), our problem reduces exactly to the single-machine problem studied by Mor and Shapira. Therefore, since an NP-hard problem is contained as a special case of our more general formulation, our problem is also NP-hard.

5.7 Connection to the Dynamic Programming Attempt

The baseline described above is exponential, since it explicitly examines many possible subsets of jobs.

This difficulty is closely related to the Dynamic Programming discussion in Section 3. In the rejection version of the problem, a compact DP state might have the form

$$DP[i][b][t][q],$$

where:

- i is the number of jobs considered,
- b is the rejection budget already used,
- t is the total processing time of the accepted jobs,
- q is the maximum processing time among the accepted jobs.

However, as discussed earlier, even without rejection we did not find a way to justify such a compact state for the general multi-machine problem, because the optimal order is not known in advance. With rejection, the difficulty increases further: not only is the optimal order unknown, but the set of accepted jobs is also unknown.

We emphasize again that this does not prove that no efficient DP algorithm exists. It is possible that another structural property or another state representation could lead to such an algorithm. However, within the scope of this project, we did not find a polynomial or pseudo-polynomial DP that guarantees an optimal solution for the multi-machine scheduling problem with rejection.

5.8 Need for Heuristic Algorithms

Since the exact baseline enumerates all possible rejection subsets, it cannot be used for large instances.

Therefore, in order to handle larger inputs, we developed three polynomial-time heuristic algorithms. These heuristics do not examine all possible rejection subsets. Instead, they select jobs for rejection according to different criteria involving contribution, savings, and rejection cost.

In all three heuristics, after the rejected jobs are determined, WSPT-MCI is used to schedule the remaining jobs.

These heuristics are presented in the next section. Later, we evaluate their quality by comparing them against the exact baseline on small instances, and by comparing them against each other on larger instances.

6 Heuristic Algorithms for the Rejection Problem

6.1 Motivation

In the previous section, we defined the scheduling problem with job rejection and introduced an exact baseline based on enumerating all possible rejection sets. This baseline is suitable only for small instances, since the number of possible rejection sets is exponential in the number of jobs.

Therefore, in order to handle larger instances, we developed three polynomial-time heuristic algorithms. These heuristics do not examine all possible rejection sets. Instead, they select jobs for rejection according to different criteria involving contribution, savings, and rejection cost.

The goal of the heuristics is to obtain high-quality solutions with a running time significantly lower than that of the exact baseline.

6.2 Common Scheduling Subroutine: WSPT-MCI

All three heuristics use the same scheduling subroutine: WSPT-MCI.

Each heuristic first decides which jobs should be rejected. Then, the set of non-rejected jobs is scheduled using WSPT-MCI.

The reason for using WSPT-MCI is that it gives an optimal sequence for any given set of jobs in the proportionate flow shop without rejection. Therefore, after a heuristic selects the set of jobs to be processed, this set can be scheduled optimally with respect to the scheduling subproblem.

In other words, the difference between the heuristics is not in the way the non-rejected jobs are scheduled, but in the way the rejected jobs are selected.

6.3 H1: Contribution-Ratio Heuristic

The first heuristic, H1, is the simplest and fastest of the three.

The idea is to estimate the contribution of each job to the objective function in the initial schedule, and compare this contribution to the rejection cost of the job.

First, all jobs are scheduled using WSPT-MCI. Then, for each job, we compute its contribution to the objective function:

$$contribution_j = w_j C_j.$$

Next, a score is computed for each job:

$$score_j = \frac{contribution_j}{e_j},$$

where e_j is the rejection cost of job j.

If $e_j = 0$, then rejecting the job does not consume any rejection budget. In this case, if the contribution is positive, the score is treated as very large, giving priority to rejecting jobs that are free to reject.

A job with a high score is considered a good candidate for rejection, since it contributes a large amount to the objective function relative to its rejection cost.

The jobs are sorted in decreasing order of this score. The heuristic then scans the sorted list and rejects jobs as long as the rejection budget allows it.

After the rejected jobs are selected, the remaining jobs are scheduled again using WSPT-MCI.

6.3.1 Pseudocode

```
Input: jobs, rejection budget R

initialSchedule = WSPT-MCI(jobs)

for each job j in initialSchedule:
    compute completion time C_j
    contribution[j] = w_j * C_j
```

```
    if e_j = 0:
        score[j] = infinity
    else:
        score[j] = contribution[j] / e_j

sort jobs by decreasing score

rejected = empty set
rejectUsed = 0

for each job j in sorted jobs:
    if rejectUsed + e_j <= R:
        add j to rejected
        rejectUsed = rejectUsed + e_j

accepted = jobs not in rejected
finalSchedule = WSPT-MCI(accepted)

return finalSchedule, rejected
```

The advantage of H1 is that it is simple and very fast. Its disadvantage is that it is based only on the initial contribution of each job, and does not recompute the effect of previous rejection decisions.

6.4 H2: One-Shot Savings Heuristic

The second heuristic, H2, attempts to measure more directly the savings obtained by rejecting a specific job.

First, the cost of scheduling all jobs is computed:

$$baseCost.$$

Then, for each job j, the algorithm checks what the schedule cost would be if only job j were rejected and all other jobs were scheduled using WSPT-MCI.

The saving of job j is defined as

$$saving_j = baseCost - cost_without_j.$$

Next, a score is computed:

$$score_j = \frac{saving_j}{e_j}.$$

A job with a high score is considered attractive for rejection, since removing it significantly decreases the objective function relative to its rejection cost.

If the saving is not positive, the job is not considered a good candidate for rejection. If $e_j = 0$ and the saving is positive, the score is treated as very large.

After computing the score for each job, the jobs are sorted in decreasing order of score. The heuristic then rejects jobs as long as the rejection budget allows it.

Finally, the non-rejected jobs are scheduled again using WSPT-MCI.

6.4.1 Pseudocode

```
Input: jobs, rejection budget R

baseSchedule = WSPT-MCI(jobs)
baseCost = cost(baseSchedule)

for each job j in jobs:
    trialJobs = jobs without j
    trialSchedule = WSPT-MCI(trialJobs)
    trialCost = cost(trialSchedule)

    saving[j] = baseCost - trialCost

    if saving[j] <= 0:
        score[j] = 0
    else if e_j = 0:
        score[j] = infinity
    else:
        score[j] = saving[j] / e_j

sort jobs by decreasing score

rejected = empty set
rejectUsed = 0

for each job j in sorted jobs:
    if saving[j] > 0 and rejectUsed + e_j <= R:
        add j to rejected
        rejectUsed = rejectUsed + e_j

accepted = jobs not in rejected
finalSchedule = WSPT-MCI(accepted)

return finalSchedule, rejected
```

The advantage of H2 is that it measures the actual saving obtained by removing each job individually, rather than relying only on its initial contribution. Its disadvantage is that the computation is still one-shot: the saving of each job is computed relative to the schedule containing all jobs, and is not updated after rejection decisions are made.

6.5 H3: Iterative Greedy with Local Search

The third heuristic, H3, is the most complex of the three. It is designed to improve solution quality through iterative greedy decisions and a local search phase.

Unlike H1 and H2, where a score is computed for each job only once, H3 recomputes the effect of rejecting a job at every step.

The algorithm starts with all jobs accepted and no rejected jobs. At each iteration, the algorithm considers every currently accepted job and evaluates what would happen if that job were rejected.

For each possible rejection candidate, it computes:

$$newCost = costafterremovingjobj,$$

$$saving = currentCost - newCost,$$

and

$$score = \frac{saving}{e_j}.$$

Among all jobs that can be rejected without exceeding the rejection budget, the algorithm selects the job with the highest score.

After rejecting a job, the schedule cost of the remaining jobs is recomputed using WSPT-MCI, and the algorithm continues to the next iteration.

The greedy phase stops when no job can be rejected within the remaining budget while producing a positive saving.

After the greedy phase, the algorithm performs a local search phase. In this phase, it checks one-for-one swaps of the following form:

$$returnonerejectedjobandrejectonecurrentlyacceptedjobinstead.$$

If such a swap improves the objective function and still satisfies the rejection budget, the best improving swap is applied.

At the end of the algorithm, the remaining accepted jobs are scheduled using WSPT-MCI.

6.5.1 Pseudocode

```
Input: jobs, rejection budget R

accepted = all jobs
rejected = empty set
rejectUsed = 0

currentSchedule = WSPT-MCI(accepted)
currentCost = cost(currentSchedule)
```

```
while true:

    bestJob = none
    bestScore = 0

    for each job j in accepted:

        if rejectUsed + e_j > R:
            continue

        trialAccepted = accepted without j
        trialSchedule = WSPT-MCI(trialAccepted)
        trialCost = cost(trialSchedule)

        saving = currentCost - trialCost

        if saving <= 0:
            continue

        if e_j = 0:
            score = infinity
        else:
            score = saving / e_j

        if score > bestScore:
            bestJob = j
            bestScore = score
            bestTrialCost = trialCost

    if bestJob = none:
        break

    move bestJob from accepted to rejected
    rejectUsed = rejectUsed + e_bestJob
    currentCost = bestTrialCost

bestSwap = none

for each rejected job r:
    for each accepted job a:

        newRejectUsed = rejectUsed - e_r + e_a

        if newRejectUsed > R:
            continue
```

```
        trialAccepted = accepted
        remove a from trialAccepted
        add r to trialAccepted

        trialSchedule = WSPT-MCI(trialAccepted)
        trialCost = cost(trialSchedule)

        if trialCost < currentCost:
            save this swap as bestSwap if it is the best found

if bestSwap exists:
    apply bestSwap

finalSchedule = WSPT-MCI(accepted)

return finalSchedule, rejected
```

The advantage of H3 is that it adapts the rejection decisions to the current state at every iteration, and therefore is expected to produce higher-quality solutions. In addition, the local search phase can correct some of the mistakes made during the greedy phase.

The disadvantage is its higher running time. Moreover, since the local search checks only one-for-one swaps, the algorithm may still get stuck in a local solution that is not globally optimal.

6.6 Complexity of the Heuristics

In this section, we analyze the asymptotic running time of each heuristic, using WSPT-MCI as an $O(n^2)$ scheduling subroutine.

For H1, the algorithm computes one initial schedule, computes a score for each job, sorts the jobs, and finally schedules the non-rejected jobs again. Therefore, the time complexity is

$$O(n^2).$$

For H2, the algorithm checks, for each job, what the cost would be if that job were rejected. Each such check applies WSPT-MCI to a set of size at most n. Therefore, the time complexity is

$$O(n^3).$$

For H3, at each greedy iteration many candidate rejections are checked, and each check requires recomputing a schedule. In addition, the algorithm performs a local search phase over possible one-for-one swaps. Under our implementation, the resulting time complexity is

$$O(n^4).$$

Thus, the asymptotic running times are:

$$H1 : O(n^2), \qquad H2 : O(n^3), \qquad H3 : O(n^4).$$

As the heuristic becomes more complex, it may produce better solutions, but at the cost of a higher running time.

6.7 Discussion

The three heuristics represent different trade-offs between solution quality and running time.

H1 is the simplest and fastest heuristic. It is useful when running time is very important or when a fast heuristic reference point is needed.

H2 provides a more direct estimate of the effect of rejecting a single job, since it computes the actual saving obtained by removing each job. However, it is still based on a one-shot calculation.

H3 is the strongest heuristic in terms of expected solution quality, since it recomputes the savings at each step and adds a local search phase. On the other hand, it is also the slowest heuristic.

In the experimental section, we evaluate the three heuristics in two ways:

1. For small instances, we compare them against the exact baseline.
2. For larger instances, where the exact baseline is computationally impractical, we compare the heuristics against each other and also examine their running times.

7 Experimental Setup

7.1 Purpose of the Experiments

After defining the scheduling problem with job rejection and presenting the three heuristic algorithms, the goal of the experiments was to evaluate both the solution quality and the running time of the algorithms.

The experiments were designed to answer two main questions:

1. How close are the heuristics to the exact solution on instances where an exact baseline can be computed?
2. How do the heuristics behave as the number of jobs increases, especially in terms of the trade-off between solution quality and running time?

For this purpose, the experiments were divided into two parts:

1. Quality benchmark.
2. Scalability benchmark.

7.2 Quality Benchmark: Comparison Against the Exact Baseline

In the first part of the experiments, we evaluated the quality of the heuristics by comparing them against the exact baseline.

As described in Section 5, the exact baseline enumerates all feasible rejection sets and applies WSPT-MCI to the jobs that are not rejected. Therefore, for small instances, this baseline gives an exact solution to the scheduling problem with rejection.

Since the running time of the exact baseline is exponential, this benchmark was performed only on relatively small instances.

In each experimental run, we generated

$$5000$$

random instances.

We performed

$$10$$

independent runs.

Therefore, in total, we tested

$$50000$$

random instances against the exact baseline.

For each random instance, the number of jobs was chosen from the range

$$n \in [5, 14],$$

and the number of machines was chosen from the range

$$M \in [2, 10].$$

For each job, the following values were generated:

- p_j: processing time,
- w_j: job weight,
- e_j: rejection cost.

The processing times and weights were positive values, while the rejection cost was allowed to be zero.

The rejection budget R was chosen as a ratio of the total rejection cost of all jobs. This generated instances with different budget levels, ranging from tight budgets to more generous budgets.

For each instance, four solutions were computed:

1. the exact baseline,
2. H1,
3. H2,
4. H3.

Each heuristic was then compared against the exact baseline.

7.3 Metrics in the Quality Benchmark

For each heuristic, we measured the following metrics.

7.3.1 Exact Match

The number of instances in which the heuristic produced exactly the same objective value as the exact baseline.

7.3.2 Within 5% of the Baseline

The number of instances in which the heuristic objective value was at most 5% worse than the baseline.

This metric is important because in heuristic optimization, a solution that is not exactly optimal may still be considered very good if it is close to the optimum.

7.3.3 Absolute Gap

The absolute gap between the heuristic value and the baseline value:

$$\textit{heuristic cost} - \textit{baseline cost}.$$

We measured both the average absolute gap and the maximum absolute gap.

7.3.4 Relative Gap

The relative gap, in percentage, was computed as

$$\frac{\textit{heuristic cost} - \textit{baseline cost}}{\textit{baseline cost}} \cdot 100.$$

We measured both the average relative gap and the maximum relative gap.

In addition, we computed a version of the relative gap with a 5% tolerance. In this version, if the relative gap was at most 5%, it was counted as zero for the purpose of measuring the error.

7.4 Scalability Benchmark: Comparison Between the Heuristics

In the second part of the experiments, we evaluated the behavior of the heuristics on larger instances.

For these instances, the exact baseline was not computed, because enumerating all rejection sets requires exponential time and is not practical for large numbers of jobs.

Therefore, in this benchmark, the comparison was performed only between the three heuristics.

The tested instance sizes were

$$n = 20, 50, 100, 150, 200.$$

For each value of n, in each experimental run, we generated

$$30$$

random instances.

Thus, each run contained

$$5 \cdot 30 = 150$$

scalability instances.

Since we performed 10 runs, the total number of scalability instances was

$$1500.$$

In each instance, the number of machines M was also chosen randomly, as in the quality benchmark, and the jobs were generated using the same random generation mechanism.

7.5 Metrics in the Scalability Benchmark

Since no exact baseline was computed in this benchmark, we defined the best value found among the three heuristics as

$$\textit{best observed}.$$

For each instance,

$$\textit{best observed} = \min(H1\ \textit{cost}, H2\ \textit{cost}, H3\ \textit{cost}).$$

It is important to emphasize that *best observed* is not necessarily the optimal solution. It is only the best solution found among the three heuristics.

The following metrics were measured.

7.5.1 Best Observed Counts

For each heuristic, we counted the number of instances in which it achieved the *best observed* value.

7.5.2 Within 5% of the Best Observed Value

For each heuristic, we counted the number of instances in which its value was at most 5% worse than the best observed value.

7.5.3 Average Gap from Best Observed

For each heuristic, we computed its relative gap from the best observed value:

$$\frac{\textit{heuristic cost} - \textit{best observed}}{\textit{best observed}} \cdot 100.$$

7.5.4 Gap with 5% Tolerance

As in the quality benchmark, we also computed a version in which gaps of at most 5% were counted as zero.

7.5.5 Average Running Time

In addition to solution quality, we measured the average running time of each heuristic.

This metric is especially important because the heuristics have different asymptotic running times:

$$H1 : O(n^2), \qquad H2 : O(n^3), \qquad H3 : O(n^4).$$

Therefore, significant differences in running time are expected as n increases.

7.6 Comparison Between Different Runs

Since the input instances were generated randomly, we performed multiple independent runs of the experiment in order to check that the conclusions were not dependent on a single random run.

For each run, all result files were saved in a separate folder. Then, a separate comparison program was used to combine the results from all runs and compute overall averages.

This allowed us to present both:

- the results of each individual run,
- averages over all 10 runs.

The comparison program also generated graphs that illustrate the results, including:

- accuracy compared to the exact baseline,
- average relative gap compared to the exact baseline,
- the frequency with which each heuristic was the best observed heuristic,
- average running times,
- overall averages across all runs.

7.7 Summary of the Experimental Design

In summary, the experimental design consisted of two complementary parts.

In the first part, we evaluated the quality of the heuristics against an exact baseline on small instances. This allowed us to measure how close each heuristic was to the exact solution.

In the second part, we evaluated the behavior of the heuristics on larger instances, where the exact baseline was computationally impractical. In this part, the heuristics were compared against each other, and their running times were measured.

Together, these two benchmarks provide a broader view of the algorithms: quality relative to an exact solution on small instances, and scalability on larger instances where exact optimization is not practical.

8 Experimental Results

This section presents the results obtained from the computational experiments. The experiments were divided into two parts. In the first part, the three heuristics were compared with the exact rejection-set baseline on 50,000 randomly generated instances. In the second part, larger instances were tested without the exact baseline, and the comparison was made only between the heuristics.

8.1 Comparison with the Exact Baseline

The first experiment included 10 independent runs, each consisting of 5,000 random instances. Therefore, the comparison with the exact rejection-set baseline was performed over a total of 50,000 instances.

Table 1 summarizes the percentage of instances in which each heuristic matched the baseline exactly, and the percentage of instances in which its solution was within 5% of the baseline.

The results show that H3 was clearly the most accurate heuristic. It matched the baseline in 89.19% of the tested instances, and was within 5% of the baseline in 92.42% of the instances. H1 and H2 were less accurate, with exact match rates of 69.36% and 71.18%, respectively.

Table 1: Accuracy compared with the exact rejection-set baseline

Heuristic	Exact match	Within 5%
H1	69.36%	75.91%
H2	71.18%	77.78%
H3	89.19%	92.42%

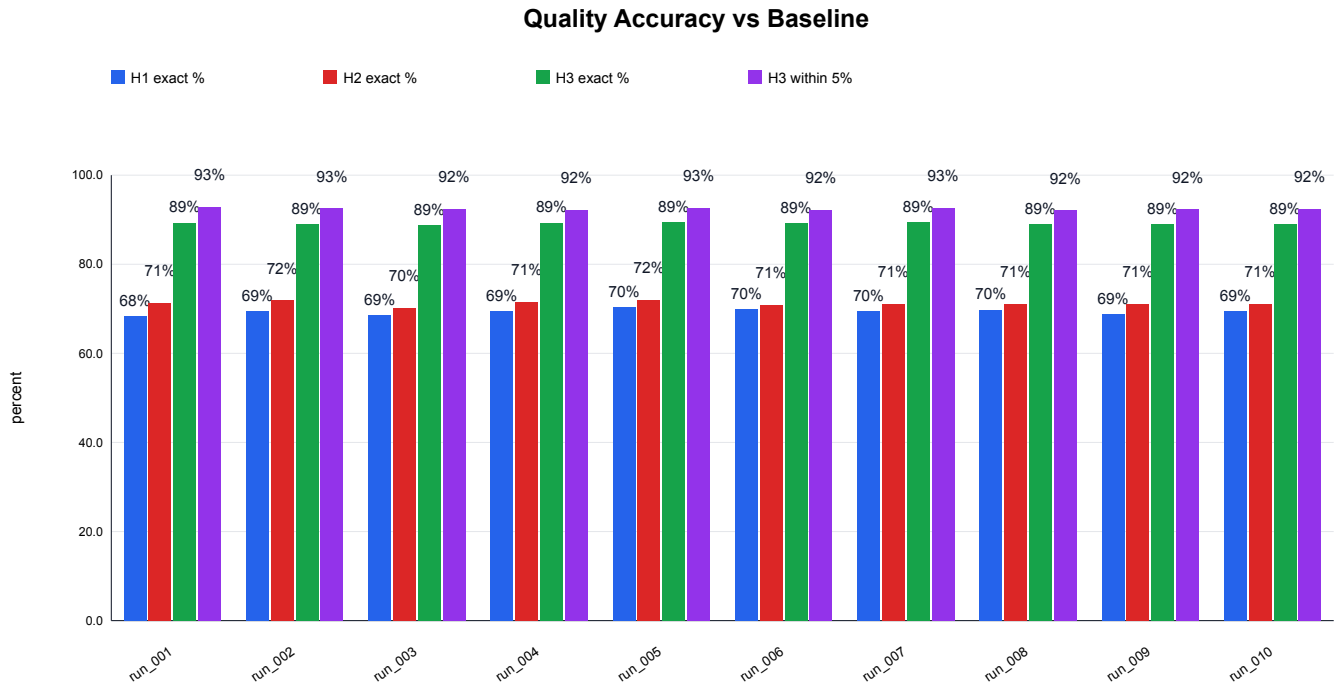


Figure 1: Quality accuracy compared with the exact rejection-set baseline

Table 2 presents the average relative gap from the exact baseline. The gap is computed as the percentage increase in the objective value of the heuristic compared with the baseline.

Table 2: Average relative gap from the exact rejection-set baseline

Heuristic	Average relative gap
H1	13.42%
H2	10.44%
H3	2.22%

After applying the 5% tolerance rule, where gaps of at most 5% are treated as acceptable, the average relative gaps were as shown in Table 3.

Table 3: Average relative gap with 5% tolerance

Heuristic	Average relative gap with 5% tolerance
H1	13.27%
H2	10.28%
H3	2.15%

These results indicate that the local improvement phase in H3 has a significant effect on solution quality. Although H3 has a higher running time than H1 and H2, it produced solutions much closer to the exact baseline.

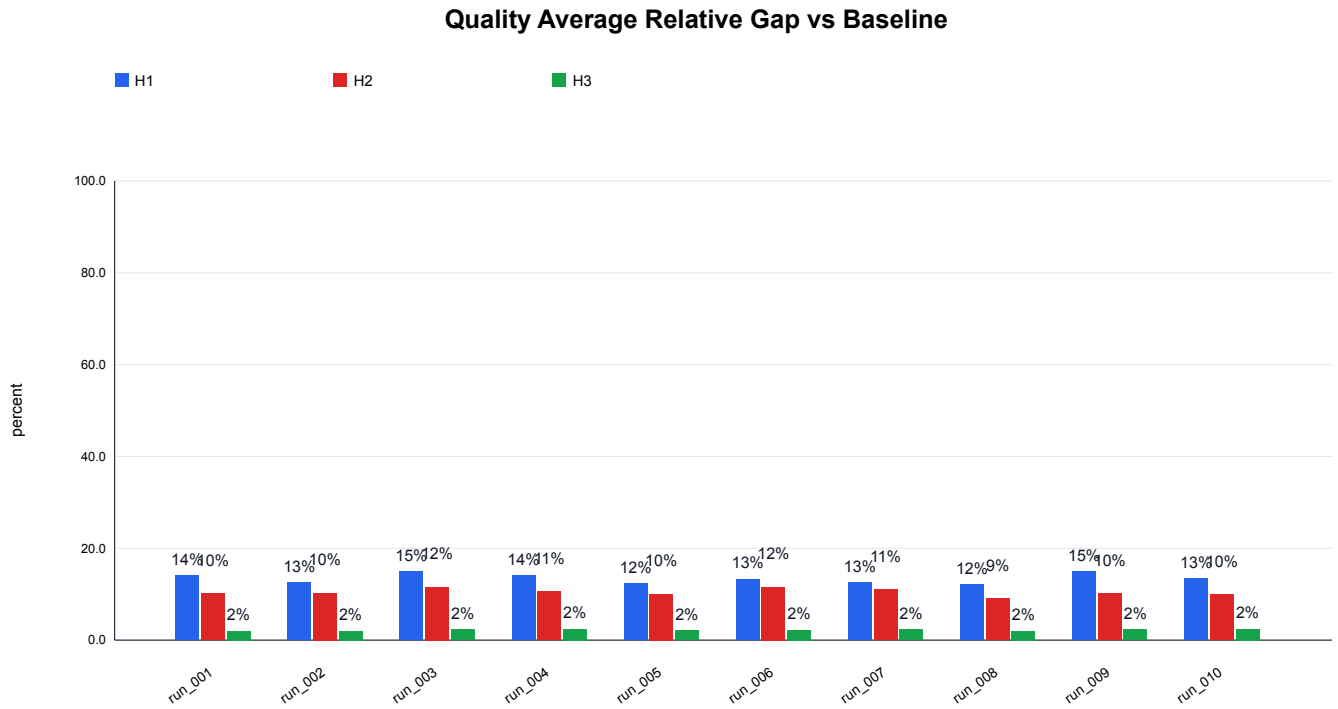


Figure 2: Average relative gap from the exact rejection-set baseline

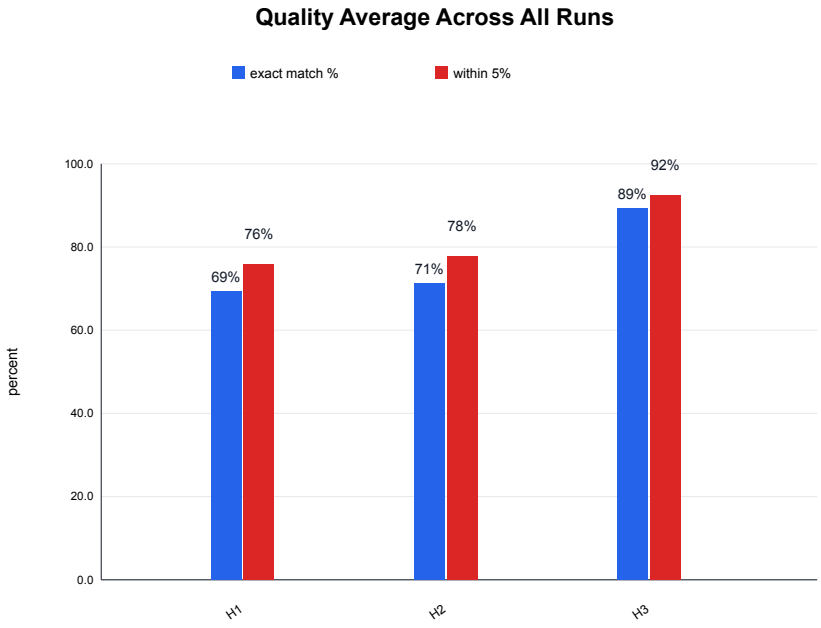


Figure 3: Average quality results across all runs

8.2 Scalability Experiments

The second part of the experiment tested larger instances, for which the exact rejection-set baseline was not computed. This was necessary because enumerating all rejection subsets requires exponential time and becomes impractical for larger values of n.

In these experiments, the heuristics were compared only against each other. For each instance, the best objective value among H1, H2, and H3 was considered the best observed result. In addition to counting exact best-observed results, we also counted the number of instances in which each heuristic was within 5% of the best observed result. In this second measure, a solution whose objective value is at most 5% larger than the best observed objective value is not treated as an error.

Across all scalability experiments, 1,500 instances were tested. Table 4 summarizes both measures.

The results show that H3 was the strongest heuristic in the scalability experiments. It achieved the best observed result in 92.33% of the tested instances, and it was within 5% of the best observed result in 99.07% of the instances. H2 achieved the best observed result in 34.73% of the instances and was within

Table 4: Best observed results in the scalability experiments

Heuristic	Best observed	Within 5% of best observed
	149/1500	299/1500
H1	9.93%	19.93%
	521/1500	1048/1500
H2	34.73%	69.87%
	1385/1500	1486/1500
H3	92.33%	99.07%

5% in 69.87% of the instances. H1 was the weakest heuristic in this comparison, achieving the best observed result in only 9.93% of the instances and being within 5% in 19.93% of the instances.

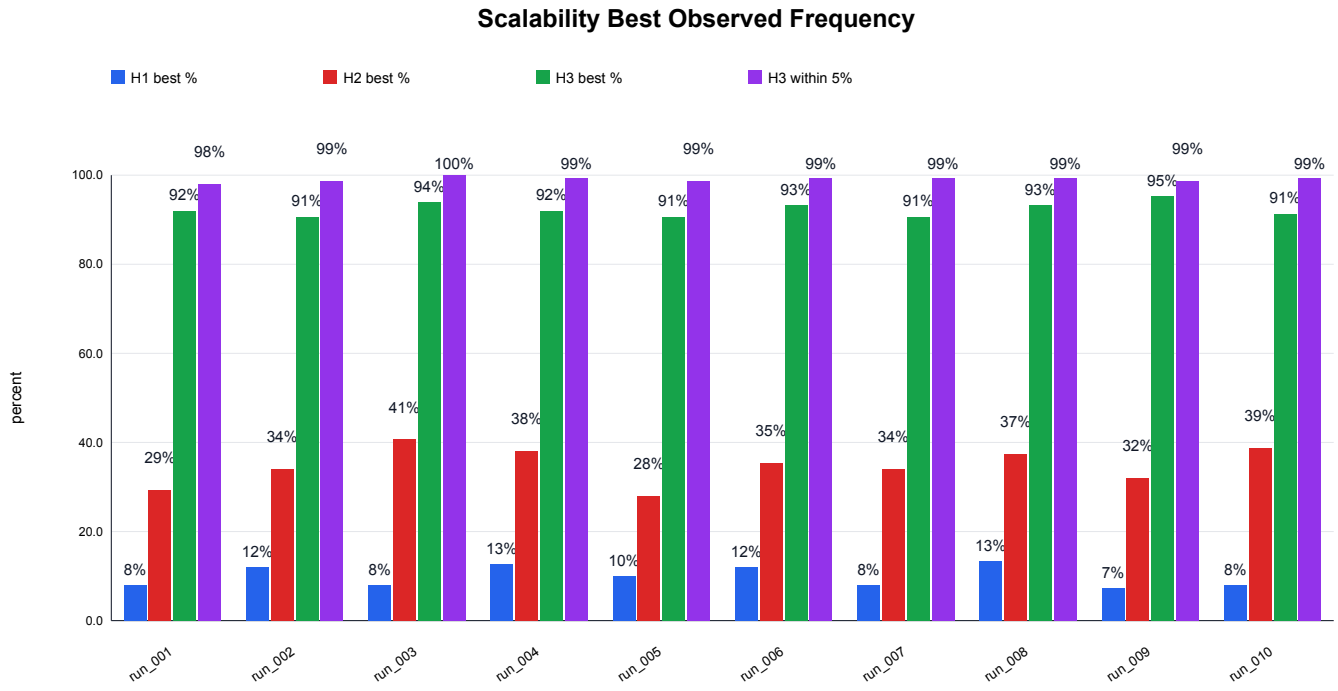


Figure 4: Best observed frequency in the scalability experiments

Table 5 presents the average gap from the best observed solution.

Table 5: Average gap from the best observed solution

Heuristic	Average gap	Average gap with 5% tolerance
H1	62.81%	62.52%
H2	9.55%	9.00%
H3	0.21%	0.17%

The scalability results again show that H3 produced the best solutions by a large margin. Its average gap from the best observed solution was only 0.21%, whereas H2 had an average gap of 9.55%, and H1 had an average gap of 62.81%.

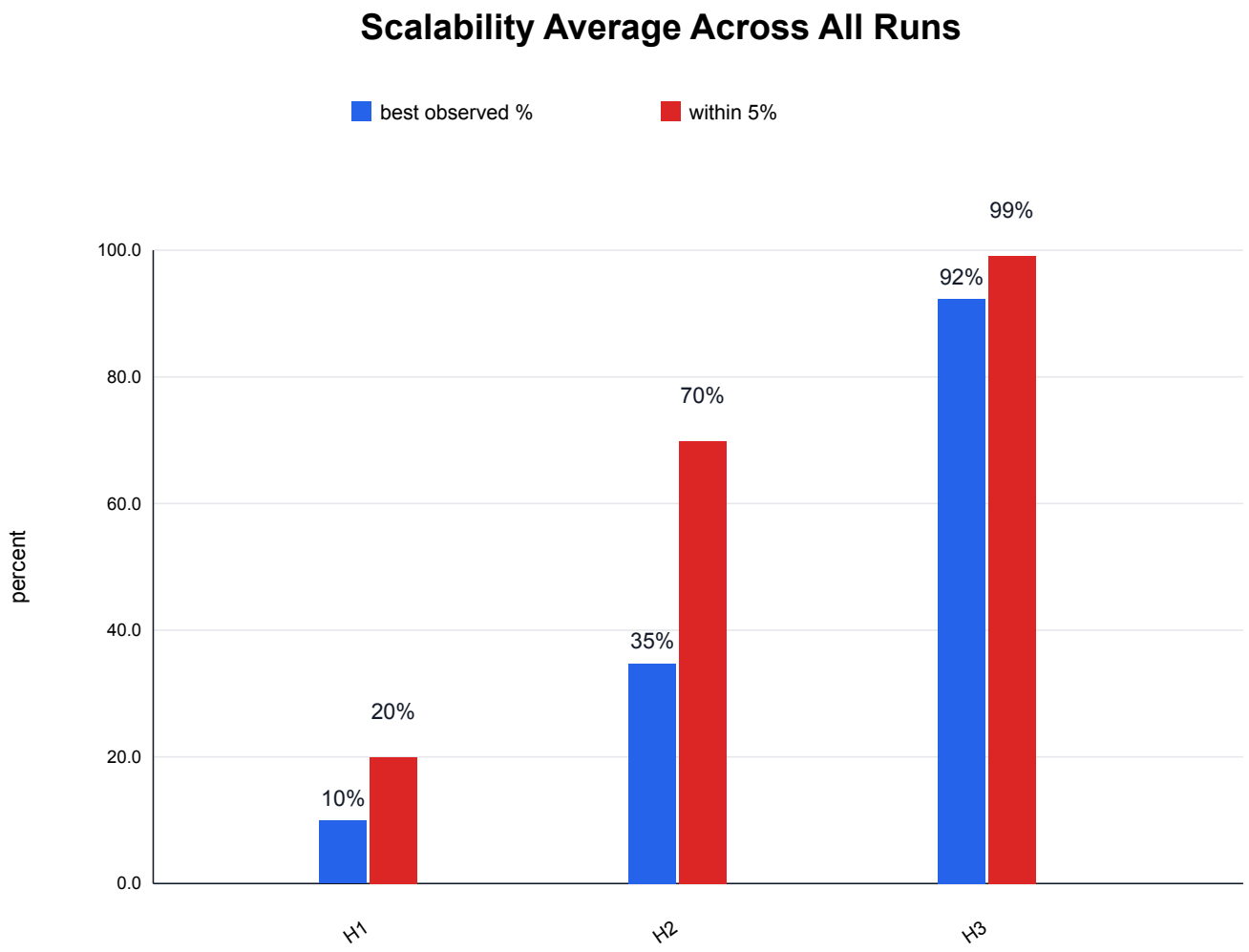


Figure 5: Average scalability results across all runs

8.3 Running Time

The running time results are summarized in Table 6. As expected, H1 was the fastest heuristic, followed by H2, while H3 required the largest running time because of its local search phase.

Table 6: Average running time in the scalability experiments

Heuristic	Average running time
H1	0.25 ms
H2	18.23 ms
H3	851.89 ms

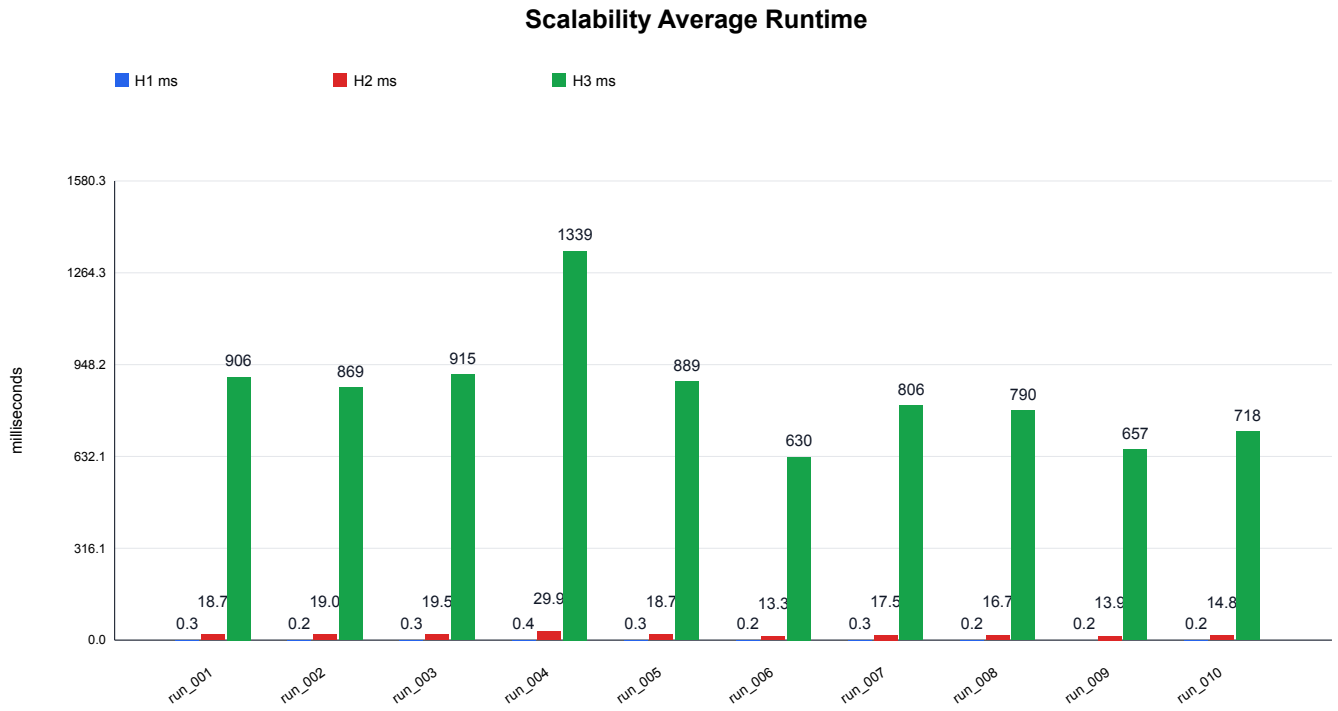


Figure 6: Average running time in the scalability experiments

8.3.1 Discussion

The results demonstrate a clear trade-off between solution quality and running time. H1 is extremely fast but often produces weaker solutions, while H2 improves solution quality while remaining relatively efficient. H3 has a higher running time, but consistently produces the best results among the tested heuristics and solutions that are very close to the optimum. This makes H3 particularly suitable for large problem instances involving many machines and jobs, where computing the exact optimal solution becomes impractical due to the large number of possible scheduling and rejection decisions. Therefore, H3 provides a strong balance between solution quality and computational efficiency, offering near-optimal results within a reasonable running time when exact optimization is computationally infeasible.